

Exact pattern matching

The Z algorithm

CMSC 423, Fall 2026



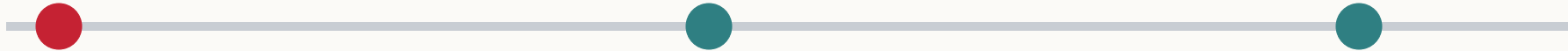
Exact matching in read mapping

A mapper first finds short exact matches between a read and the reference. Those seeds identify regions worth aligning in detail.

Find exact seeds

Filter candidates

Align candidates



Fast exact matching

Discard weak regions

Use an inexact method

**This lecture develops an exact-search component.
We'll later see that, when we have many patterns, we will want
to index them for even faster search.**

Exact pattern matching

Given a text T and a shorter pattern P , report every position where P occurs in T .

Direct comparison

Try each alignment
Worst case: $O(nm)$

Z algorithm

Reuse prefix matches
Worst case: $O(n + m)$

Later lectures will use preprocessing for repeated searches.

The exact matching problem

Given a text T and pattern P , find every substring of T equal to P .

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37
T	A	T	A	C	A	T	A	C	C	C	A	T	A	T	A	C	G	A	G	G	C	A	T	A	C	A	T	G	G	C	G	A	G	T	G	T	G	C

15

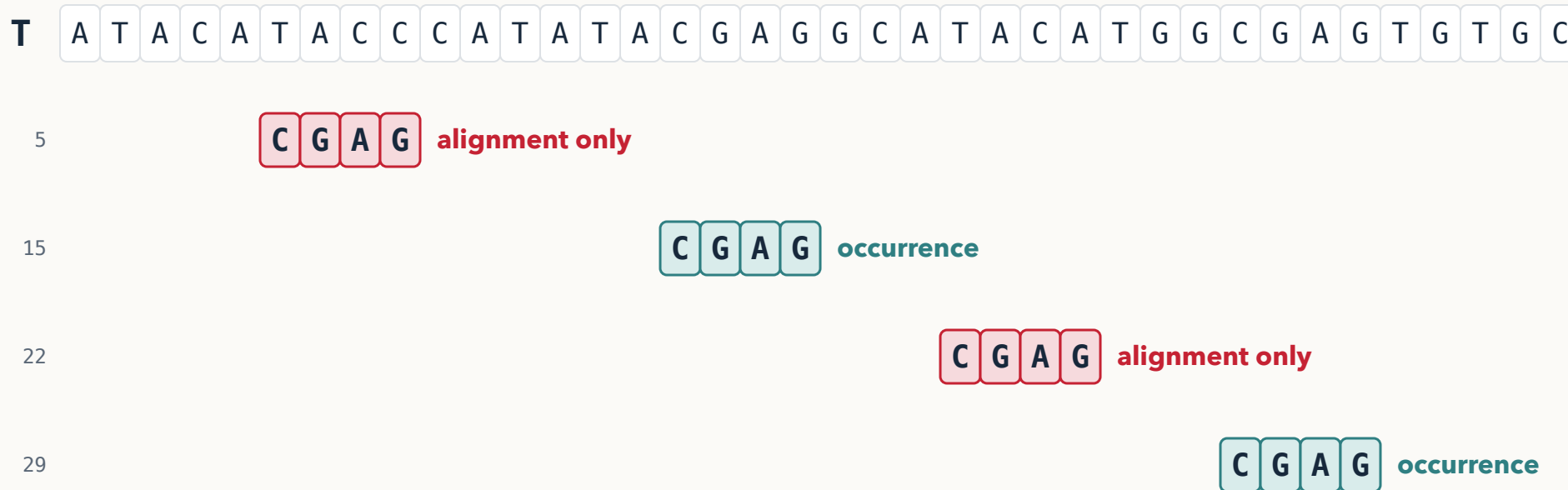
C G A G occurrence at 15

$$n = |T|, \quad m = |P|, \quad \text{and } m \leq n$$

Output: the starting index of every occurrence.

Occurrences and alignments

Every occurrence is an alignment. An alignment becomes an occurrence only when all characters match.



Two of these four sample alignments are occurrences.

Counting alignments

How many starting positions can a length-m pattern have in a length-n text?

T A T A C A T A C C C A T A T A C G A G G C A T A C A T G G C G A G T G T G C

0 C G A G start 0

1 C G A G start 1

2 C G A G start 2

3 C G A G start 3

4 C G A G start 4

...

The final alignment starts at $n - m$.

Counting alignments

Valid starting indices are

$0, 1, 2, \dots, n - m$

$n - m + 1$ alignments

Equivalently: $|T| - |P| + 1$

Naive exact matching

What is the simplest correct exact-matching algorithm?

Naive exact matching

What is the simplest correct exact-matching algorithm?

Try every alignment. Compare left to right and stop at the first mismatch.

Naive exact matching

Check each possible alignment and report the matches.

```
def naive(T, P):  
    n = len(T)  
    m = len(P)  
    occurrences = []  
    for i in range(n - m + 1):  
        if P == T[i:i + m]:  
            occurrences.append(i)  
    return occurrences
```

Worst-case running time

```
def naive(T, P):  
    n = len(T)  
    m = len(P)  
    occurrences = []  
    for i in range(n - m + 1):  
        if P == T[i:i + m]:  
            occurrences.append(i)  
    return occurrences
```

What is the worst-case running time?

$n = |T|$ $m = |P|$

Naive running time

```
def naive(T, P):  
    n = len(T)  
    m = len(P)  
    occurrences = []  
    for i in range(n - m + 1):  
        if P == T[i:i + m]:  
            occurrences.append(i)  
    return occurrences
```

Outer loop
At most $n - m + 1 = O(n)$
alignments

**Comparison at one
alignment**
At most m character
checks

Worst case: $O(n) \cdot O(m) = O(nm)$

Best-case behavior

T G A G A G G A G T T A T A T A T G A A T A G A G A T A G A G A C G A G

0 C G A G fails immediately

31

C G A G one match

Most alignments fail on the first character, so each costs $O(1)$. One complete match costs $O(m)$.

Best case: $O(n + m)$

Worst-case behavior



Each alignment matches the first $m - 1$ characters and fails only at the last character.

Worst case: $\Theta(nm)$

Closing the gap

Best case

$O(n + m)$

Worst case

$O(nm)$

Can we guarantee $O(n + m)$ time in the worst case?

The Z algorithm does so by reusing prefix matches.

The Z-array

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
S	A	T	A	C	G	G	C	A	C	A	T	A	C	C	A	T	A	C	G	A	A	T	A	T	A	C	A	A	A

$Z[i]$ is the length of the longest substring starting at i that matches a prefix of S .

For $i = 9$, $S[9..12] = \text{ATAC}$ matches $S[0..3]$, so $Z[9] = 4$.

A complete Z-array

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
S	A	T	A	C	G	G	C	A	C	A	T	A	C	C	A	T	A	C	G	A	A	T	A	T	A	C	A	A	A
Z	29	0	1	0	0	0	0	1	0	4	0	1	0	0	5	0	1	0	0	1	3	0	4	0	1	0	1	1	1

Examples

$Z[9] = 4$ because ATAC matches the prefix

$Z[14] = 5$ because ATACG matches the prefix

$Z[20] = 3$ because ATA matches the prefix

$Z[i] = 0$ when $S[i]$ differs from $S[0]$.

Computing the Z-array naively

At each i , compare $S[i..]$ with $S[0..]$ until the first mismatch.

$i = 2$

prefix	A...
suffix	AC...

$Z[2] = 1$

$i = 9$

prefix	ATAC...
suffix	ATAC...

$Z[9] = 4$

$i = 14$

prefix	ATACG...
suffix	ATACG...

$Z[14] = 5$

**Independent comparisons may repeat the same character work,
giving $\Theta(n^2)$ time in the worst case.**

From the Z-array to matching

S	A	T	A	C	G	G	C	A	C	A	T	A	C	C	A	T	A	C	G	A	A	T	A	T	A	C	A	A	A
Z	29	0	1	0	0	0	0	1	0	4	0	1	0	0	5	0	1	0	0	1	3	0	4	0	1	0	1	1	1

Computing each Z value independently costs $\Theta(n^2)$.

Before improving that computation, how can Z values solve exact pattern matching?

We need prefix matches against a pattern, not only against the text itself.

Pattern and text

For every position i in T , measure how many characters of P match $T[i..]$.

P A C A

T A T A C G G C A C A T A C C A T A C G A A T A T A C A A A

7

A C A pattern match

Place P before T so ordinary prefix matches answer this question.

Matching with P\$T

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
S	A	C	A	\$	A	T	A	C	G	G	C	A	C	A	T	A	C	C	A	T	A	C	G	A	A	T	A	T	A	C	A	A	A
Z	33	0	1	0	1	0	2	0	0	0	0	3	0	1	0	2	0	0	1	0	2	0	0	1	1	0	1	0	3	0	1	1	1

For $S = P\$T$, every position with $Z[i] = |P|$ marks an occurrence of P in T .

Convert a position i in S to position $i - |P| - 1$ in T .

\$ is a separator that does not occur in P or T .

Why the separator matters

Stops matches

A match cannot continue from P into T across \$.

Caps Z values

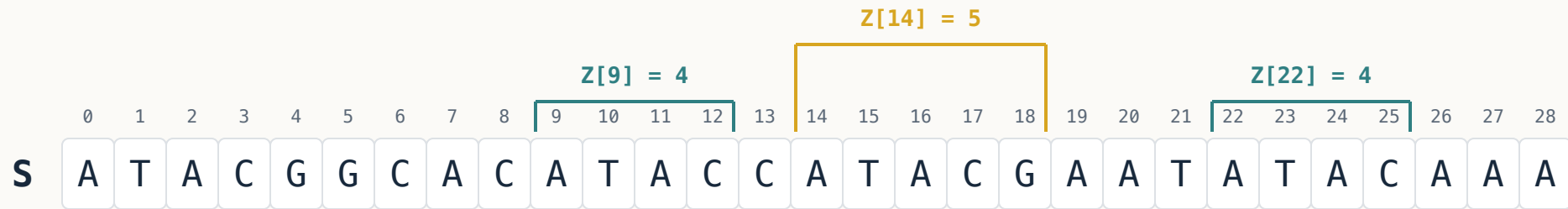
For positions in T, $Z[i] \leq |P|$.

Identifies occurrences

$Z[i] = |P|$ exactly when P matches at that text position.

The remaining task is to compute all Z values without repeating comparisons.

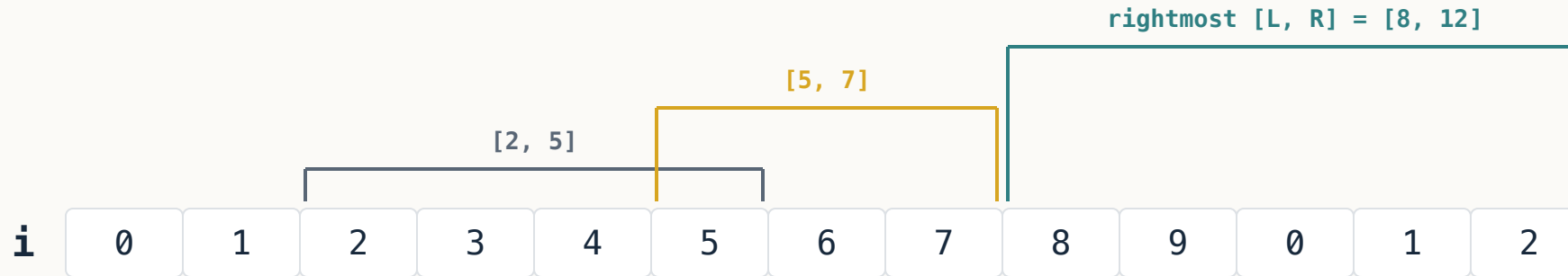
Z-boxes



The Z-box at i is $[i, i + Z[i] - 1]$. Every character in that interval matches the corresponding prefix character.

No box is drawn when $Z[i] = 0$.

The rightmost Z-box invariant



At step k , the algorithm keeps the Z-box with the largest right endpoint R among positions already processed.

If k lies inside $[L, R]$, a **mirrored** prefix value often determines $Z[k]$ without any character comparisons.

Z algorithm in motion

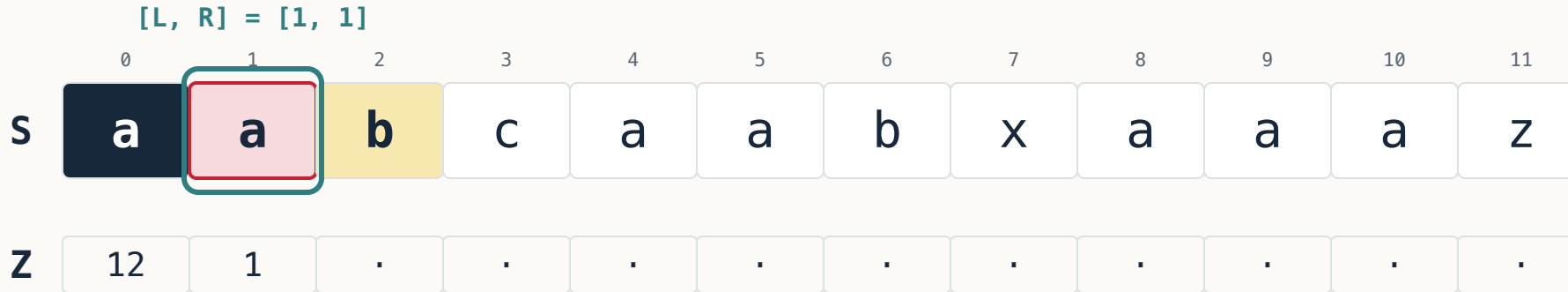
	0	1	2	3	4	5	6	7	8	9	10	11
s	a	a	b	c	a	a	b	x	a	a	a	z
z	12	.	.	.	.	.	.	.	.	.	.	.

Compute $Z[k]$ from left to right

$Z[k]$ measures how many characters starting at k match the prefix. The algorithm remembers the rightmost interval $[L, R]$ already known to match the prefix.

The next frames separate explicit comparison, copying, and boundary extension.

Outside the current Z-box

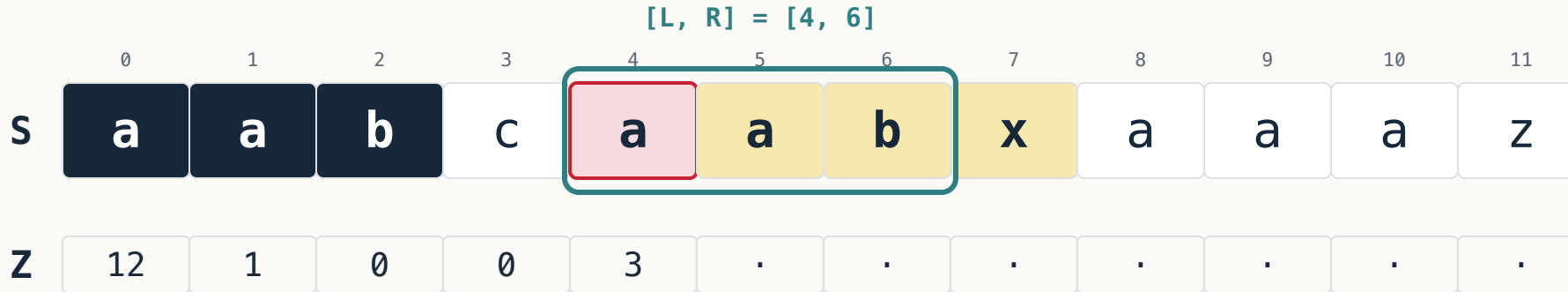


k = 1 lies beyond R

$S[1]$ matches $S[0]$. The next comparison fails, so $Z[1] = 1$ and the new rightmost box is $[1, 1]$.

Outside the box, the algorithm compares characters explicitly.

A new rightmost Z-box

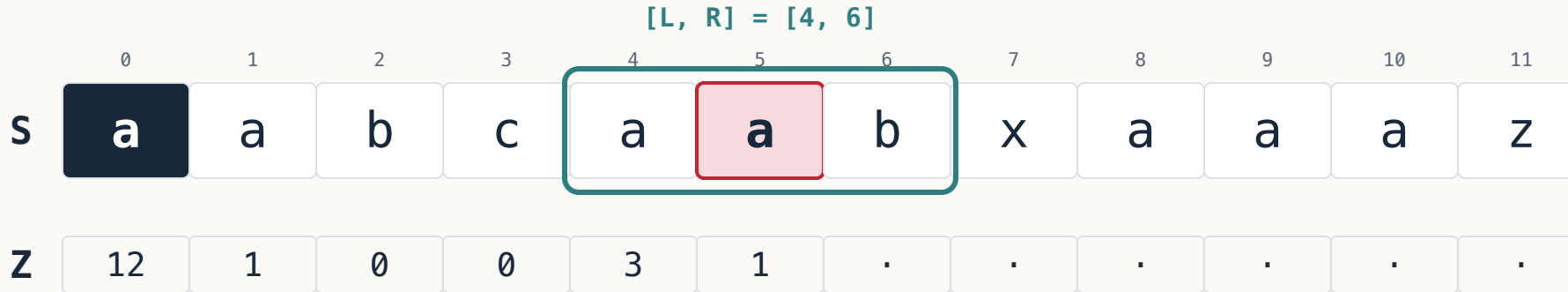


$k = 4$ also lies beyond R

$S[4..6] = \text{aab}$ matches the prefix $S[0..2]$. The next characters differ, so $Z[4] = 3$ and $[L, R]$ becomes $[4, 6]$.

The box records three matched characters for later reuse.

Copying a value inside the box

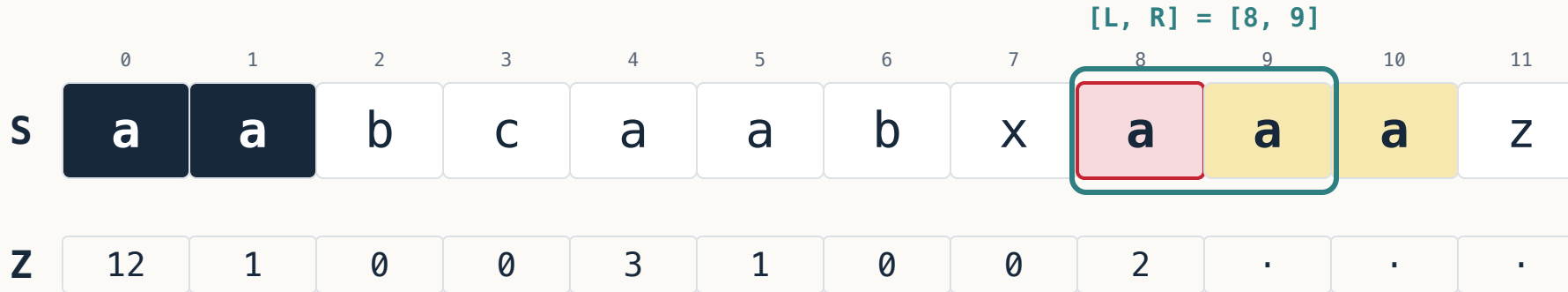


$k = 5$ lies inside $[4, 6]$

Mirror k to $k' = k - L = 1$. Since $Z[1] = 1$ is smaller than the two characters remaining in the box, copy $Z[5] = 1$.

No character comparison is needed.

A second rightmost Z-box

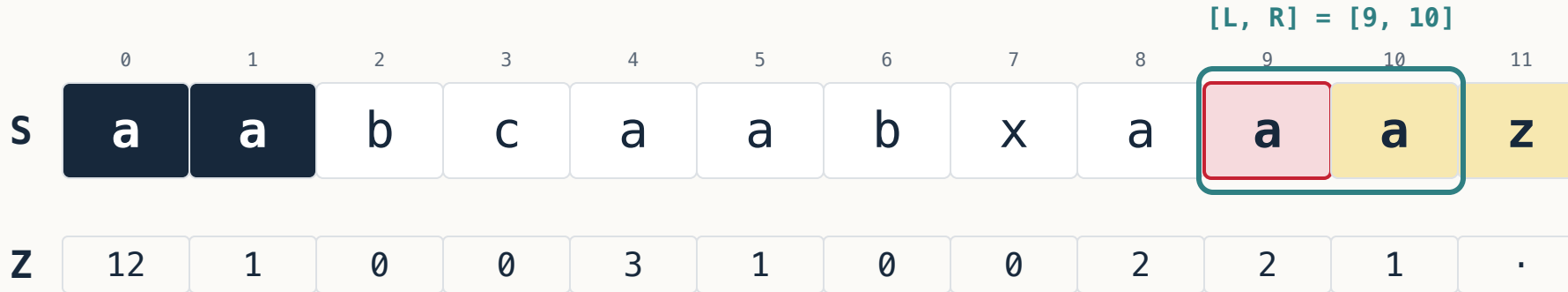


$k = 8$ lies beyond R

The first two characters match the prefix, then $S[10]$ differs from $S[2]$. Thus $Z[8] = 2$ and $[L, R]$ becomes $[8, 9]$.

Each successful explicit match moves R to the right.

Extending beyond the box



The copied match reaches R

At $k = 9$, $Z[1]$ reaches the box boundary. One comparison beyond R succeeds, so $Z[9] = 2$ and the box extends to $[9, 10]$.

Only a boundary-reaching copy can require more comparisons.

Why the Z algorithm is linear

	0	1	2	3	4	5	6	7	8	9	10	11
s	a	a	b	c	a	a	b	x	a	a	a	z
z	12	1	0	0	3	1	0	0	2	2	1	0

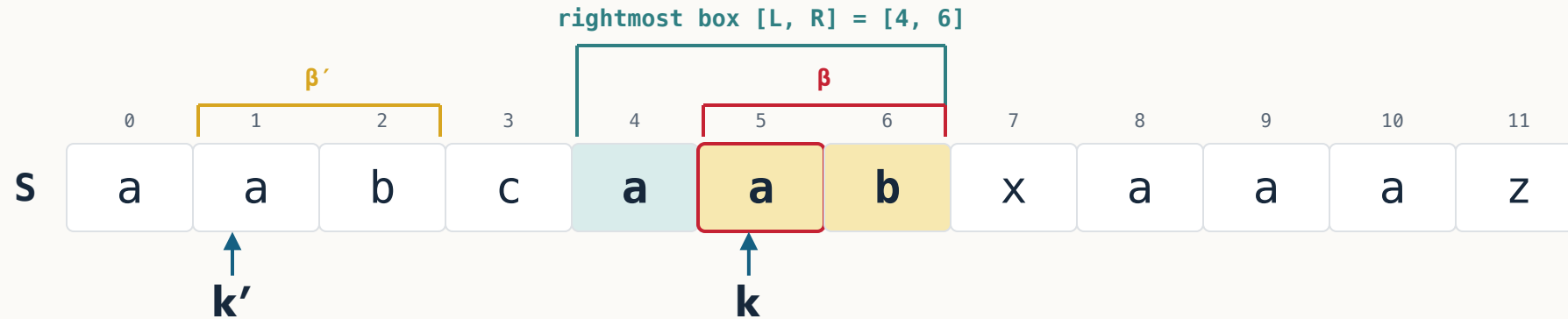
Final Z-array

R never moves left. Each successful explicit comparison advances R, and each iteration contributes at most one failed comparison.

All Z values are computed in $O(n)$ time.

State and notation for the case analysis

Before computing $Z[k]$, all values $Z[1..k-1]$ are correct and $[L, R]$ is the processed Z-box with the largest right endpoint.



mirror

$k' = k - L$; position k' in the prefix mirrors position k in the box.

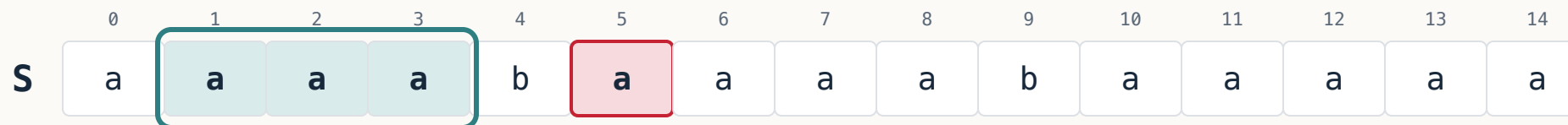
known piece

$\beta = S[k..R]$, $b = |\beta| = R - k + 1$, and $\beta' = S[k'..k'+b-1]$.

Because $[L, R]$ matches the prefix, $\beta = \beta'$. The cases ask how $Z[k']$ compares with b .

Case 1: k lies outside the Z-box

rightmost Z-box $[L, R] = [1, 3]$



$k = 5$ lies to the right of $R = 3$



current position $k = 5$ $Z[k] = ?$

Rule

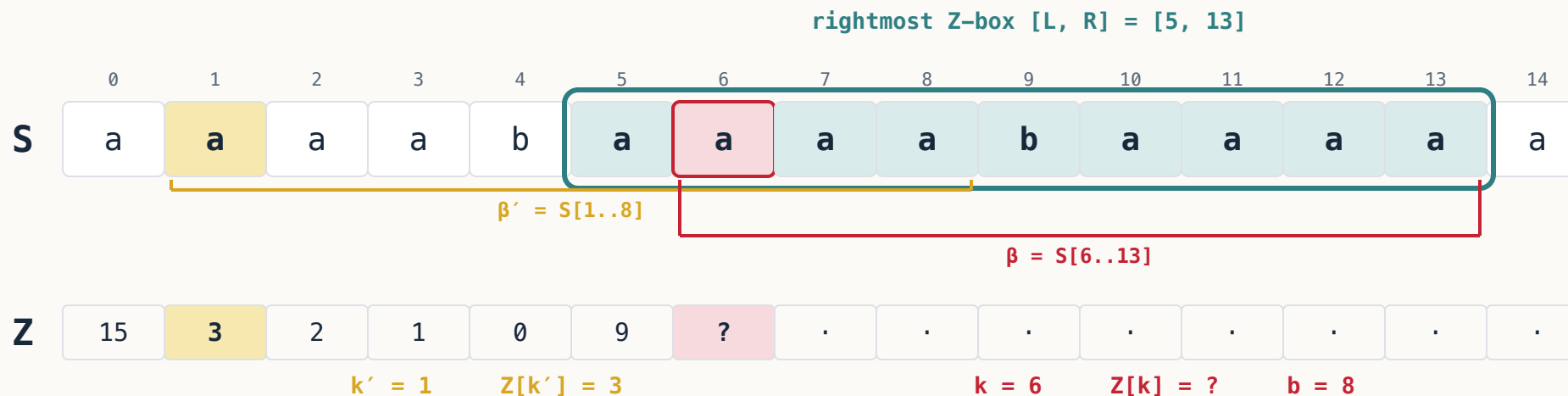
Compare $S[k..]$ with the prefix $S[0..]$ until the first mismatch. The number of successful comparisons is $Z[k]$.

If $Z[k] > 0$, set $L = k$ and $R = k + Z[k] - 1$. If $Z[k] = 0$, keep the current box.

$S[5..13] = \text{aaaabaaaa}$ matches $S[0..8]$. The next comparison fails, so $Z[5] = 9$ and the new box is $[5, 13]$.

Every character in the match is checked explicitly, so the computed $Z[k]$ is exact.

Case 2a: the mirrored match ends before R



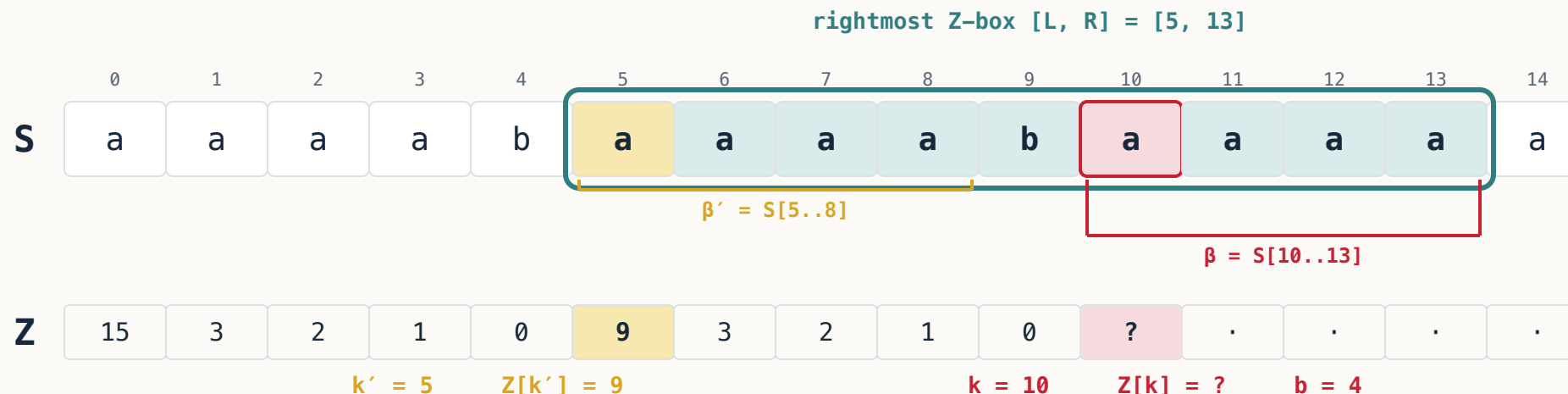
Rule Let $q = Z[k']$. When $q < b$, the mismatch that ends $Z[k']$ lies inside β' .

Since $\beta = \beta'$, the same q characters match at k and the same mismatch follows at offset q.

$$S[k+q] = S[k'+q] \neq S[q]$$

$Z[k] = Z[k'] = q$. The box $[L, R]$ stays unchanged.

Case 2b: the mirrored match crosses R



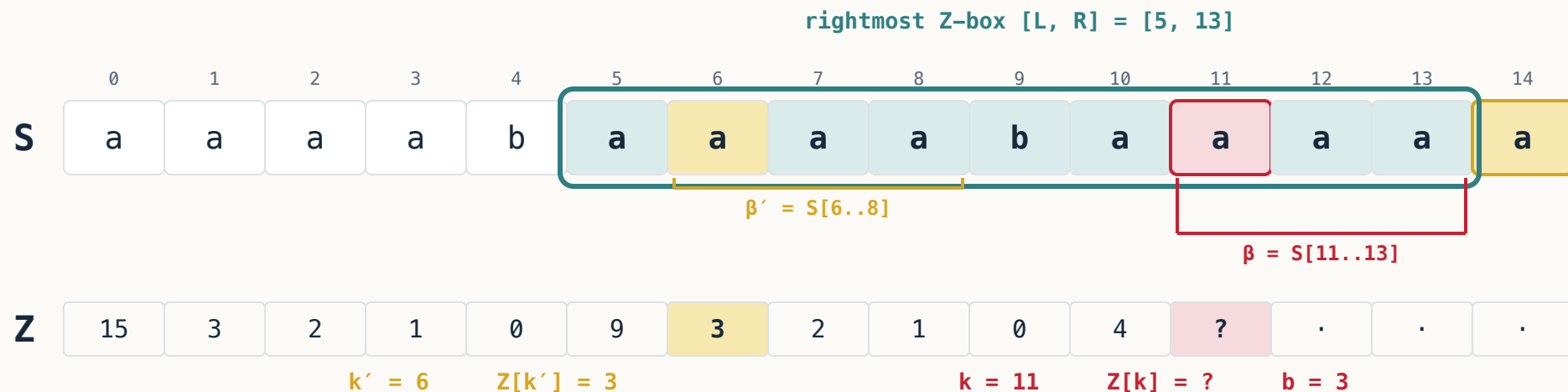
Rule **When $Z[k'] > b$, $\beta = \beta'$ already certifies b matching characters at k .**

If the match at k continued beyond b , the Z-box beginning at L would also continue beyond R . That contradicts the right endpoint of the current box.

Here $Z[k'] = Z[5] = 9 > b = 4$, but $S[R+1] = S[14] = a$ differs from $S[b] = S[4] = b$.

$Z[k] = b$. The box $[L, R]$ stays unchanged.

Case 2c: the mirrored match reaches R exactly



Rule

When $Z[k'] = b$, the box certifies b matching characters but provides no information after R .

Start explicit comparisons at $S[R+1]$ against $S[b]$. Continue until the first mismatch, then set $L = k$ and $R = k + Z[k] - 1$.

Here $S[R+1] = S[14] = a$ matches $S[b] = S[3] = a$, so $Z[11]$ extends from 3 to 4.

Only this equality case can extend the box while $k \leq R$.

Why the case algorithm is correct

Inductive invariant: before step k , every earlier Z value is exact and $[L, R]$ is the earlier Z -box with maximum R .

$$k > R$$

Explicit comparisons compute the defining longest prefix match.

$$Z[k'] < b$$

The mirrored mismatch lies inside β and transfers unchanged.

$$Z[k'] > b$$

A longer match would contradict maximality of the current box.

$$Z[k'] = b$$

The known b characters are reused; only the unknown suffix is compared.

The cases are exhaustive, each returns the exact $Z[k]$, and the update preserves the invariant. Therefore all Z values are correct by induction.

Why the Z algorithm runs in linear time

Charge work to movement of R and to the current position k.

R moves only to the right



successful comparisons

Every success outside the box or in Case 2c advances R. Since $R \leq n-1$, there are $O(n)$ successes.

failed comparisons

Each k causes at most one terminating mismatch, so there are at most $n-1$ failures.

copied values

Cases 2a and 2b use $O(1)$ arithmetic and no comparisons.

$$O(n) \text{ loop work} + O(n) \text{ comparisons} = O(n)$$

Exact matching in linear time

Problem

Report every occurrence of P in T .

Direct method

Worst-case time $O(nm)$.

Z algorithm

Compute $Z(P\$T)$ and scan for values equal to $|P|$ in $O(n + m)$.

Worked Z-array example

	positions 0-18																		
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
S	A	G	A	C	T	T	\$	A	C	A	C	G	A	G	A	C	A	T	T
Z	38	0	1	0	0	0	0	1	0	1	0	0	4	0	1	0	1	0	0

positions 19-37																			
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
S	A	T	A	G	A	G	A	C	T	T	A	G	A	T	T	A	G	G	C
Z	1	0	3	0	6	0	1	0	0	0	3	0	1	0	0	2	0	0	0

$Z[23] = 6 = |P|$, so $P = \text{AGACTT}$ occurs in T at position $23 - 7 = 16$.

Conclusion

- The Z-algorithm lets us compute occurrences of pattern P in text T in $O(n+m)$ time.
- The Z-algorithm was *not* the first linear time exact pattern matching algorithm, but several others can be interpreted in terms of the Z-algorithm.
- If you explore other exact pattern matching algorithms (Knut Morris Pratt), one can view the pattern matching table pm as a special case of applying the Z-algorithm.